

DataLab 实验报告

一、实验信息

项目	Lab1
实验标题	DataLab：整数与浮点数的位级操作
姓名	徐明
学号	25303050254

二、实验目标

在限定的运算符、运算次数和数据类型范围内实现整数与 IEEE 754 单精度浮点数相关操作；通过操作检查脚本和测试程序验证实现是否符合题目要求。

二、运行结果

```
mingxu@LAPTOP-D640PIU7:~/DataLab$ make clean
make all
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
mingxu@LAPTOP-D640PIU7:~/DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 5/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 16/16 operators
roundEvenPow2: 19/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 39/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 45/52 operators
floatScaleThreeHalves: 57/60 operators
floatRoundEven: 26/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图一：终端运行 ./check_ops.py bits.c 的截图

```
mingxu@LAPTOP-D640PIU7:~/DataLab$ ./btest
Score Rating Errors Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

图二：终端运行 ./btest 的截图

四、各函数实现思路

(1) 整数位操作

P1.signMask: 提取输入的符号信息，并将符号位扩展成题目要求的掩码形式。

P2.bitXor: 利用德摩根律和按位取反、按位与等允许操作组合出异或。

P3.negativePart: 根据输入符号构造全零或全一掩码，再筛选出题目要求的负数部分。

P4.copyBitWithin: 分别构造源位和目标位的掩码，读取源位后清除目标位原值，再将源位写入目标位置。

P5.logicalShift: 先进行右移，再构造低位全 1、高位全 0 的掩码，清除算术右移产生的符号扩展。

P6.swapNibblePairs: 构造每个字节低四位的掩码，将低四位左移四位、将高四位右移四位后合并。

P7.secondLowestZeroBit: 先通过按位取反把零位转成 1，再利用隔离最低位及移除最低位的位运算，定位第二个最低位。

P8.oddParity: 通过逐级异或折叠各位，得到所有位的奇偶性。

P9.rotateRightBits: 将输入拆为右移部分和左移回绕部分，并用掩码屏蔽不需要的位后合并。

P10.roundEvenPow2: 依据目标精度分离保留部分与舍弃部分；比较舍弃部分和半单位，恰好平局时检查保留结果的奇偶性以实现 ties-to-even。

P11.midpointTowardFirst: 计算两个端点的中点时避免直接相加造成有符号溢出，并按题目规定处理无法整除时的取整方向。

P12.isBetweenEitherOrder: 分别处理两个端点的大小关系，判断输入是否落在端点构成的区间内。

P13.mul5Sat: 用移位和加法计算 $5 \times x$ ，通过输入范围/符号关系检测上溢或下溢；溢出时依据输入符号返回对应饱和值。

P14.classifyAdd3: 根据题目要求对三数相加的结果分类；通过符号位及边界关系判断类别，避免仅依赖可能已溢出的结果。

(2) 浮点数位操作

P15.floatScaleThreeHalves: 拆分符号、指数和尾数，分别处理规格化数、非规格化数、零和特殊值；对有效数执行乘 3 并调整指数以实现乘以 $3/2$ ，舍入时采用最近偶数规则。

P16.floatRoundEven: 按指数判断数值的整数位和小数位，比较被舍弃部分与半个单位；平局时依据整数部分奇偶性决定是否进位，并保留负零及 NaN、无穷大输入的规定行为。

P17.float_i2f: 对整数取无符号幅值，找到最高有效位以确定浮点指数，再形成 24 位有效数；若需舍弃低位，则按最近偶数规则舍入，并处理舍入导致的有效数进位。

其他位级操作

P18.bitCount: 采用并行位计数方法，先对相邻位分组求和，再逐步合并为更大的组，最终得到 32 位整数中 1 的总数。

P19.bitReverse: 通过掩码和移位，依次交换相邻 1 位、2 位、4 位、8 位及 16 位分组，完成 32 位顺序反转。

五、结果分析

`./check_ops.py bits.c` 显示 19 个函数均通过运算符检查，没有函数超过规定的运算数限制。

`./btest` 的总分为 110/110，各项错误数均为 0，说明截图所示测试输入下，函数实现通过了测试。